

Titans: Learning to Memorize at Test Time

NeurIPS 2025

Fundamental Team
Presenter : Hyeju Shin

1. Introduction

Previous Researches

: Transformer

- State-of-the-art models in sequence modeling due to in-context learning and ability to learn at scale.
- Model direct dependencies via attention, but **time and memory complexity grows as $O(N^2)$** .
- ⇒ To overcome the scalability issue, recent studies aim to design different variants of **linear Transformers** (softmax → kernel function).

: Linear Transformer

- Used to enhance scalability and efficiency (linear vs. quadratic complexity), whose advantages is appeared for very long context.
- ⇒ On the other hand, a very long context **cannot be properly compressed in a small vector-valued or matrix-valued state**.

: Most existing architecture (ranging from Hopfield Networks to LSTMs and Transformer)

- face challenges when dealing with **generalization, length extrapolation, and/or reasoning**, all of which are parts of real-world tasks.

Problem

: Although these architectures draw inspiration from the human brain, each of which are missing

- (1) a crucial component for learning process—such as short-term memory, long-term memory, meta- memory
- (2) how these components are interconnected systems that can operate independently
- (3) the ability to actively learn from data and memorize the abstraction of past history.

1. Introduction

Research Question

- : (Q1) What is a proper memory update mechanism?
- : (Q2) What is a good memory retrieval process?
- : (Q3) How to design an efficient architecture that incorporates different interconnected memory modules
- : (Q4) Is a deep memory module needed to effectively store/remember long past?

1. Introduction

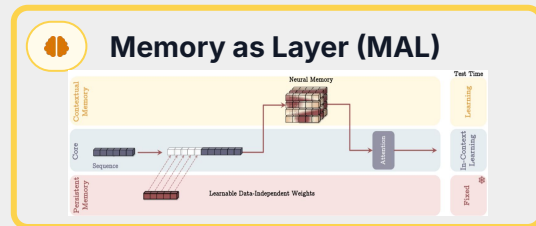
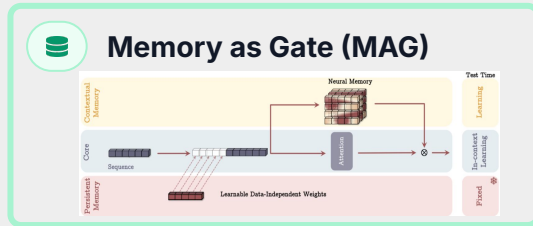
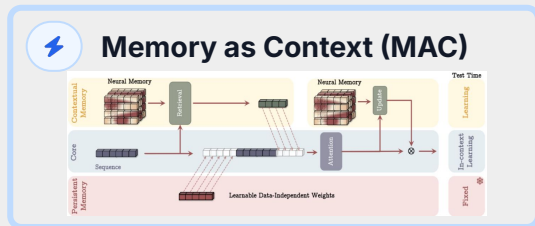
Research Question

- : (Q1) What is a proper memory update mechanism?
- : (Q2) What is a good memory retrieval process?
- : (Q3) How to design an efficient architecture that incorporates different interconnected memory modules
- : (Q4) Is a deep memory module needed to effectively store/remember long past?



Titans: Learning to Memorize at Test Time

- : **(Neural Memory)** a (deep) neural long-term memory that learns how to memorize/store the data into its parameters at test time.
 - a decaying mechanism that consider the proportion of memory size and the amount of data surprise.
- : **(Titans Architectures)** After designing the long-term neural memory, Titans is designed.
 - (1) **Core**: this module consists of the short-term memory, and is responsible for the main flow of processing the data
 - (2) **Long-term Memory**: this branch is our neural long-term memory module that is responsible to store/remember long past
 - (3) **Persistent Memory**: this is a set of learnable but data-independent parameters that encodes the knowledge about a task



1. Introduction: Preliminaries

Notations

- : $\mathbf{x} \in \mathbb{R}^{N \times d_{in}} \rightarrow$ input
- : $\mathbf{M} \rightarrow$ neural network (neural network module)
- : $\mathbf{Q}, \mathbf{K}, \mathbf{V} \rightarrow$ query, key, value
- : $\mathbf{S}_j^{(i)} \rightarrow$ i-th segment
- : $\mathbf{N}(\mathbf{x}), \mathbf{N}^*(\mathbf{x}) \rightarrow$ forward pass with weight adjustment, without weight adjustment
- : $\mathbf{N}^{(k)} \rightarrow$ k-th layer of the neural network

Attentions

$$\cdot \quad \mathbf{Q} = \mathbf{x}\mathbf{W}_{\mathbf{Q}}, \quad \mathbf{K} = \mathbf{x}\mathbf{W}_{\mathbf{K}}, \quad \mathbf{V} = \mathbf{x}\mathbf{W}_{\mathbf{V}}, \quad \mathbf{y}_i = \sum_{j=1}^i \frac{\exp(\mathbf{Q}_i^{\top} \mathbf{K}_j / \sqrt{d_{in}}) \mathbf{V}_j}{\sum_{\ell=1}^i \exp(\mathbf{Q}_i^{\top} \mathbf{K}_{\ell} / \sqrt{d_{in}})},$$

Efficient Attentions

$$\cdot \quad \mathbf{y}_i = \sum_{j=1}^i \frac{\phi(\mathbf{Q}_i^{\top} \mathbf{K}_j)}{\sum_{\ell=1}^i \phi(\mathbf{Q}_i^{\top} \mathbf{K}_{\ell})} \mathbf{V}_j = \sum_{j=1}^i \frac{\phi(\mathbf{Q}_i)^{\top} \phi(\mathbf{K}_j)}{\sum_{\ell=1}^i \phi(\mathbf{Q}_i)^{\top} \phi(\mathbf{K}_{\ell})} \mathbf{V}_j = \frac{\phi(\mathbf{Q}_i)^{\top} \sum_{j=1}^i \phi(\mathbf{K}_j) \mathbf{V}_j}{\phi(\mathbf{Q}_i)^{\top} \sum_{\ell=1}^i \phi(\mathbf{K}_{\ell})},$$

choosing the kernel as identity matrix $\Rightarrow \mathbf{M}_t = \mathbf{M}_{t-1} + \mathbf{K}_t^{\top} \mathbf{V}_t, \mathbf{y}_t = \mathbf{Q}_t \mathbf{M}_t,$

$$\Rightarrow \mathbf{M}_i = (\mathbf{K}_1 \mathbf{V}_1 + \mathbf{K}_2 \mathbf{V}_2 + \cdots + \mathbf{K}_{i-1} \mathbf{V}_{i-1}) + \mathbf{K}_i \mathbf{V}_i$$

Modern Linear Models and Their Memory Perspective

$$\cdot \quad \begin{aligned} \mathbf{M}_t &= f(\mathbf{M}_{t-1}, \mathbf{x}_t), \\ \mathbf{y}_t &= g(\mathbf{M}_t, \mathbf{x}_t), \end{aligned}$$

Write Operation
Read Operation

2. Learning to Memorize at Test Time

: Goal) overcome the lack of long-term memory and to enable the model to learn, forget, and retrieve information.

→ a neural long-term memory module, which is a meta models that learns to memorize at test time.

2.1 Long-term Memory

: To design a neural long-term memory, a model that can encode the abstraction of the past history into its parameters is needed.

→ However, memorization has been known as undesirable phenomena in neural networks (limits generalization, poor performance).

⇒ an online meta-model that learns how to memorize/forget the data at test time.

Learning Process and Surprise Metric

$$\begin{aligned} \text{ : } \quad \mathcal{M}_t &= \mathcal{M}_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t) \quad \longrightarrow \quad \mathcal{M}_t = \mathcal{M}_{t-1} + S_t, \quad S_t = \eta_t \underbrace{S_{t-1}}_{\text{Past Surprise}} - \theta_t \underbrace{\nabla \ell(\mathcal{M}_{t-1}; x_t)}_{\text{Momentary Surprise}}. \end{aligned}$$

Objective

$$\text{ : } \quad \ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(\mathbf{k}_t) - \mathbf{v}_t\|_2^2$$

Forgetting Mechanism

$$\text{ : } \quad \mathcal{M}_t = (1 - \alpha_t) \mathcal{M}_{t-1} + S_t, \quad S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t),$$

2. Learning to Memorize at Test Time

: Goal) overcome the lack of long-term memory and to enable the model to learn, forget, and retrieve information.

→ a neural long-term memory module, which is a meta models that learns to memorize at test time.

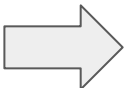
2.1 Long-term Memory

: To design a neural long-term memory, a model that can encode the abstraction of the past history into its parameters is needed.

→ However, memorization has been known as undesirable phenomena in neural networks (limits generalization, poor performance).

⇒ an online meta-model that learns how to memorize/forget the data at test time.

Learning Process and Surprise Metric

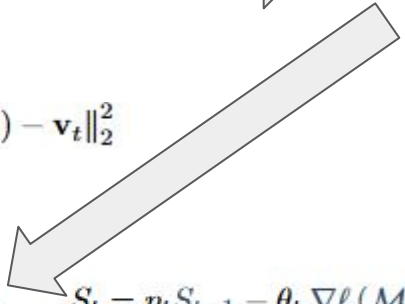
: $\mathcal{M}_t = \mathcal{M}_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t)$  $\mathcal{M}_t = \mathcal{M}_{t-1} + S_t, \quad S_t = \eta_t \underbrace{S_{t-1}}_{\text{Past Surprise}} - \theta_t \underbrace{\nabla \ell(\mathcal{M}_{t-1}; x_t)}_{\text{Momentary Surprise}}.$

Objective

: $\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(\mathbf{k}_t) - \mathbf{v}_t\|_2^2$

Forgetting Mechanism

: $\mathcal{M}_t = (1 - \alpha_t)\mathcal{M}_{t-1} + S_t, \quad S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t),$



2. Learning to Memorize at Test Time

2.2 Persistent Memory

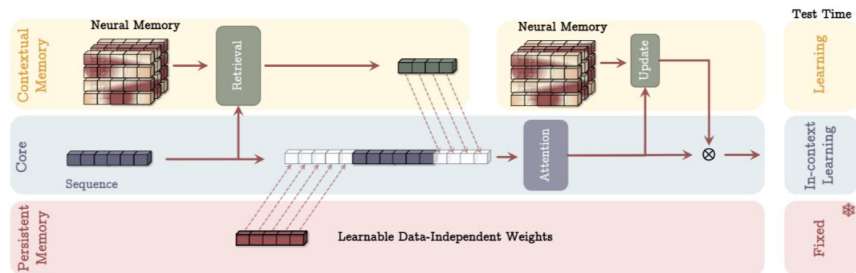
: Long-term memory can also be seen as a contextual memory, meaning that the output is fully depend on the context.

→ Therefore, a set of learnable($P = [p_1 \ p_2 \ \dots \ p_{N_p}]$) but input-independent parameters to act as task-related memory is needed.

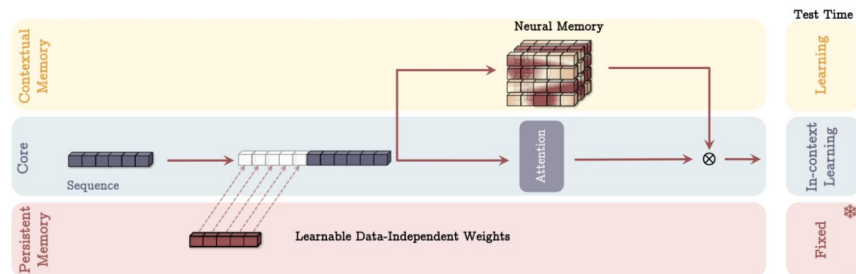
$$x_{\text{new}} = [p_1 \ p_2 \ \dots \ p_{N_p}] \parallel x,$$

3. Titans Variants: MAC, MAG, MAL

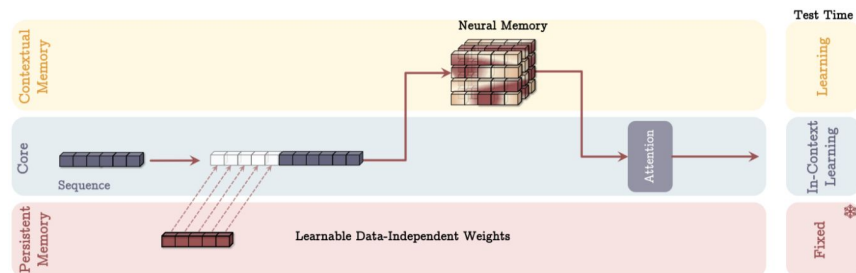
3.1 Memory as a Context (MAC)



3.2 Memory as a Gate (MAG)



3.3 Memory as a Layer (MAL)

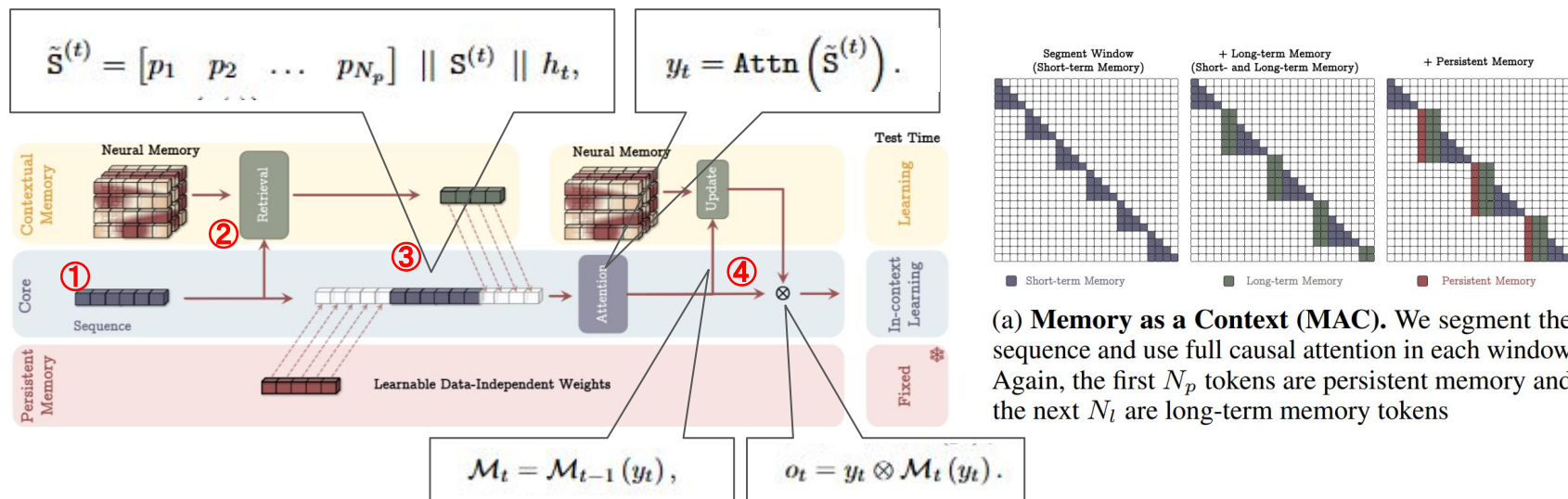


3. Memory Incorporating

3.1 Memory ad a Context (MAC)

: In the first architecture design, the memory is treated as a context to the current information.

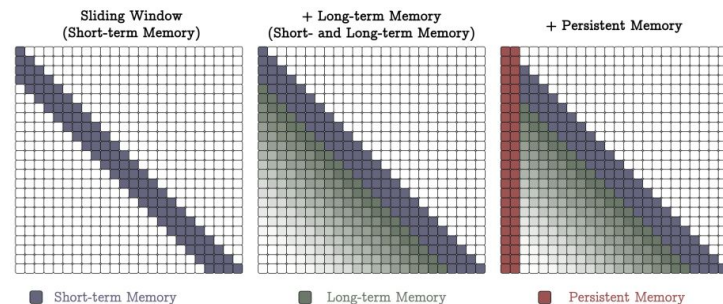
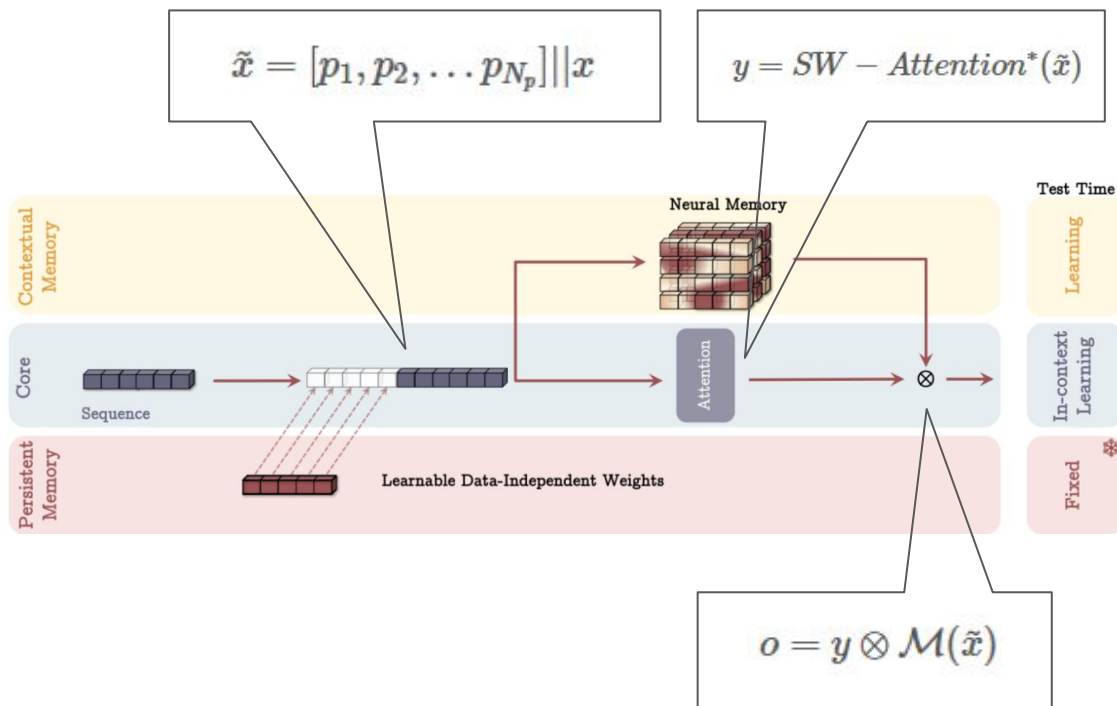
- (1) Divide a given a long sequence $x \in \mathbb{R}^{N \times d_{in}}$ into fixed-size segments $S(i)$ for $i = 1, \dots, N/C$.
- (2) Retrieve the past information that corresponds to $S^{(t)}$ as $h_t = M_{t-1}(q_t)$, where $q_t = S(t)W_Q$.
- (3) use this historical information along with persistent memory parameters(P) as the input sequence to the attention module:
- (4) then use y_t to update the long-term memory module for the next segment and the final output.



3. Memory Incorporating

3.2 Memory as a Gate (MAG)

: In this design, sliding window attention acts as a precise short-term memory, while the neural memory module is acting as a fading memory.
→ This architecture design can also be seen as a multi-head architecture where the structure of heads are different.

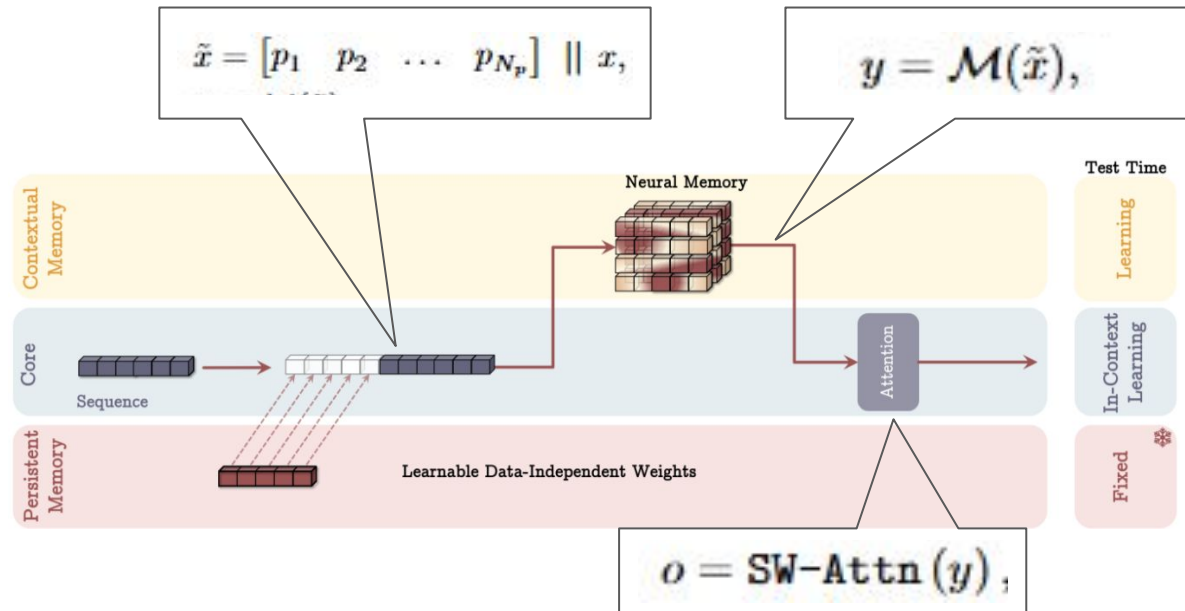


(b) **Memory as Gating (MAG).** We use sliding window attention (SWA) as a short-term memory and our neural memory module as a long-term memory, combining by a gating.

3. Memory Incorporating

3.3 Memory as a Layer (MAL)

: The last variant uses the neural Memory As a Layer (MAL) of a deep neural network.



3.4 Memory Without Attention (LMM)

: Although in the above, one simple variant of MAL is to treat neural memory module (LMM) as a sequence model without any attention.

4. Experiments

4.1 Experimental Setup

: Four variants of Titans, which we refer to as:

→ Titans with (1) Memory as Context (MAC), (2) Memory as Gate (MAG) (3) Memory as Layer (MAL) (4) neural memory module alone.

: For each of these models, four scales with: (i) 170M, (ii) 340M, (iii) 400M, (iv) 760M, and (v) 1.3B parameters are considered.

: While the first three are trained on 15B tokens sampled from FineWeb-Edu dataset, the last two are trained on 30B and 100B tokens.

4.2 Language Modeling (1/2)

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg. ↑
760M params / 30B tokens											
Transformer++	25.21	27.64	35.78	66.92	42.19	51.95	60.38	32.46	39.51	60.37	48.69
Mamba	28.12	23.96	32.80	66.04	39.15	52.38	61.49	30.34	37.96	57.62	47.22
DeltaNet	24.37	24.60	37.06	66.93	41.98	50.65	64.87	31.39	39.88	59.02	48.97
TTT	24.17	23.51	34.74	67.25	43.92	50.99	64.53	33.81	40.16	59.58	47.32
Gated DeltaNet	21.18	22.09	35.54	68.01	44.95	50.73	66.87	33.09	39.21	59.14	49.69
Samba*	20.63	22.71	39.72	69.19	47.35	52.01	66.92	33.20	38.98	61.24	51.08
Gated DeltaNet-H2*	19.88	20.83	39.18	68.95	48.22	52.57	67.01	35.49	39.39	61.11	51.49
Titans (LMM)	20.04	21.96	37.40	69.28	48.46	52.27	66.31	35.84	40.13	62.76	51.56
Titans (MAC)	19.93	20.12	39.62	70.46	49.01	53.18	67.86	36.01	41.87	62.05	52.51
Titans (MAG)	18.61	19.86	40.98	70.25	48.94	52.89	68.23	36.19	40.38	62.11	52.50
Titans (MAL)	19.07	20.33	40.05	69.99	48.85	53.02	67.61	35.65	40.57	61.72	52.29
1.3B params / 100B tokens											
Transformer++	18.53	18.32	42.60	70.02	50.23	53.51	68.83	35.10	40.66	57.09	52.25
RetNet	19.08	17.27	40.52	70.07	49.16	54.14	67.34	33.78	40.78	60.39	52.02
Mamba2	16.56	12.56	45.66	71.87	55.67	55.24	72.47	37.88	40.20	60.13	54.89
DeltaNet	17.71	16.88	42.46	70.72	50.93	53.35	68.47	35.66	40.22	55.29	52.14
Gated DeltaNet	16.42	12.17	46.65	72.25	55.76	57.45	71.21	38.39	40.63	60.24	55.32
Samba*	16.13	13.29	44.94	70.94	53.42	55.56	68.81	36.17	39.96	62.11	54.00
Gated DeltaNet-H2*	15.91	12.55	48.76	72.19	56.88	57.77	71.33	39.07	41.91	61.55	56.18
Titans (LMM)	15.60	11.41	49.14	73.09	56.31	59.81	72.43	40.82	42.05	60.97	56.82

4. Experiments

4.2 Language Modeling (2/2)

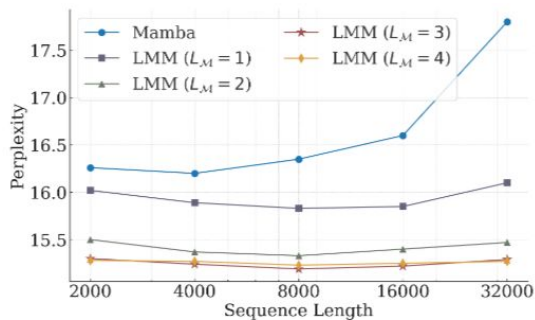
Model	S-NIAH-PK				S-NIAH-N				S-NIAH-W		
	2K	4K	8K	16K	2K	4K	8K	16K	2K	4K	8K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4
Mamba2	98.6	61.4	31.0	5.4	98.4	55.8	14.2	0.0	42.2	4.2	0.0
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6
Titans (LMM)	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8
Samba	98.8	98.0	97.4	97.2	98.8	98.6	96.2	95.6	96.8	90.0	84.0
Gated DeltaNet-H2*	99.2	97.8	97.4	98.4	98.0	97.8	96.2	95.8	97.4	96.8	88.4
Titans (MAC)	99.2	98.8	99.0	98.4	99.6	98.2	97.6	97.4	98.2	98.2	95.6
Titans (MAG)	99.4	98.0	97.4	97.4	99.2	98.8	97.2	98.6	98.0	98.0	90.2
Titans (MAL)	98.8	98.6	98.8	97.8	99.8	98.1	96.8	96.4	98.0	97.4	92.0

4.3 Needle in a Haystack

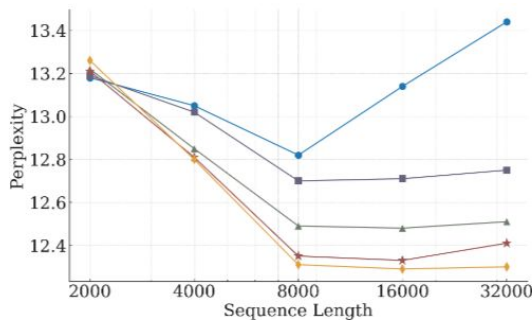
	16K	32K	64K	128K	Avg.
Transformer	81.4	79.2	63.4	49.8	68.5
Samba	83.2	80.8	69.4	34.6	67.0
LongRoPE (Phi3-3.8B)	90.8	87.7	79.8	65.3	80.9
EM-LLM	89.4	86.2	80.4	76.8	83.2
Titans (MAC)	92.0	89.2	85.2	80.4	86.7

4. Experiments

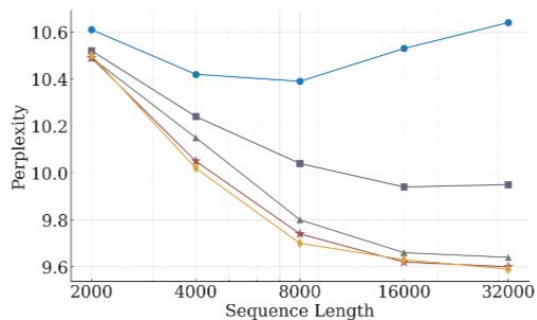
4.4 The Effect of Deep Memory



(a) 170M Parameters



(b) 360M Parameters



(c) 760M Parameters

4.5 Ablation Study

Model	Language Modeling ppl ↓	Reasoning acc ↑	Long Context acc ↑
LMM	27.01	47.83	92.68
+Attn (MAC)	26.67	48.65	97.95
+Attn (MAG)	25.70	48.60	96.70
+Attn (MAL)	25.91	47.87	96.91
Linear Memory	28.49	46.97	85.34
w/o Convolution	28.73	45.82	90.28
w/o Momentum	28.98	45.49	87.12
w/o Weight Decay	29.04	45.11	85.60
w/o Persistent Memory	27.63	46.35	92.49

5. Conclusion

Conclusion

- : presented a neural long-term memory that, as a meta in-context learner, learns to memorize at test time.
 - The neural memory module is a recurrent model in nature, and is adaptively memorizing tokens.
- : Using this memory, Titans architectures, and its three variants are suggested: the memory module as (1) a context, (2) gating, (3) a layer.
 - Experimental evaluation on diverse tasks validate that Titans are more effective than Transformers and recent modern linear recurrent.

Limitations

- : The long-term neural memory is a test time memorizer and so the in-context data is compressed into its parameters.
 - In some cases, this memorization can lead to security, safety, and alignment challenges.
 - ⇒ Future study can be to understand if the long-term memory can cause security, safety, and alignment issues.
- : The main focus of this paper has been on the design of long-term memory system.
 - ⇒ The theoretical guarantee on the capacity and memory management of Titans is saved as a future study.

Thank you for listening!